

《深度学习平台与应用》作业五（20241221）

1.列举几种常见的激活函数，并简要说明它们的作用。

2.什么是反向传播？它的核心思想是什么？

3. Numpy 实现反向传播梯度

```
import numpy as np

# 设置随机种子，方便复现
np.random.seed(42)

# 初始化参数
w1 = np.random.randn(2, 2) # 输入层到隐藏层的权重
b1 = np.random.randn(2)    # 隐藏层的偏置
w2 = np.random.randn(2)    # 隐藏层到输出层的权重
b2 = np.random.randn()     # 输出层的偏置

# 输入数据和真实标签
x = np.array([1.0, -1.5])
y_true = 0.5

# 学习率
learning_rate = 0.01

# ReLU
def relu(x):
    #todo 输入x 返回relu(x)
    pass

# ReLU 的梯度
def relu_derivative(x):
    return np.where(x > 0, 1, 0)

# 均方损失函数和其导数
def mse_loss(y_pred, y_true):
    #todo 计算均方误差
    pass

def mse_loss_derivative(y_pred, y_true):
    return y_pred - y_true # 对 y_pred 求导

# 训练过程
epochs = 100
for epoch in range(epochs):
    # 前向传播
```

```

z1 = np.dot(w1, x) + b1
h = relu(z1)
y_pred = np.dot(w2, h) + b2

# 计算损失
loss = mse_loss(y_pred, y_true)

# 反向传播
d_loss_y_pred = mse_loss_derivative(y_pred, y_true) # 损失对预测值的梯度
#todo
#d_loss_w2 = ? 对 w2 的梯度
#d_loss_b2 = ? 对 b2 的梯度

#d_loss_h =? 损失对隐藏层输出 h 的梯度
#d_loss_z1 =? 隐藏层的梯度 (通过 ReLU 计算)

#d_loss_w1 =? 对 w1 的梯度
#d_loss_b1 =? 对 b1 的梯度

# 参数更新 (随机梯度下降)
w1 -= learning_rate * d_loss_w1
b1 -= learning_rate * d_loss_b1
w2 -= learning_rate * d_loss_w2
b2 -= learning_rate * d_loss_b2

# 打印训练过程中的损失
if epoch % 10 == 0:
    print(f"Epoch {epoch}: Loss = {loss}")

# 打印最终的损失值
print(f"Final Loss: {loss}")

```

4.什么是全连接网络 (FC) 或多层感知机 (MLP) ? 它与线性分类器的主要区别是什么?

5.在梯度流的基本计算模式中, 加法门(add gate)、乘法门(mul gate)和最大值门(max gate)分别具有什么特点?